FactoryBench: Benchmarking Machine Understanding via Question-Answering

Yanis Merzouki[1,*] **Coral Izquierdo**[7] **Matei Ignuta-Ciuncanu**[3,4] **Marcos Bracamonte**[1,5]
Alessandro Lombardi[2] **Camilla Mazzoleni**[2] **Federico Martelli**[2,1] **Riccardo Maggioni**[2]
Jonas Petersen[1,2,†] **Philipp Petersen**[6,†]

[1]ETH Zurich [2]Forgis [3]Imperial College London [4]University of Berkeley [5]KTH [6]University of Vienna [7]UC3M
*Correspondence: ymerzouki@ethz.ch †Equal senior contribution

Code: github.com/Forgis-Research/FactoryBench
Dataset: huggingface.co/datasets/forgis/factorybench

Abstract

We introduce **FactoryBench**, a benchmark for evaluating time-series models and LLMs on machine understanding over academic and industrial time-series data. Q&A pairs are organised along four causal levels (state, intervention, counterfactual, decision) instantiating Pearl's ladder of causation on robotic telemetry, and span five answer formats with deterministic scoring across all of them. We propose a scalable Q&A generation framework built around structured question templates, present **FactoryWave** (a dense, multitask, multivariate sensor dataset collected from a UR3 cobot and a KUKA KR10 industrial arm at 125 and 83 Hz), and construct FactoryBench as a large-scale benchmark grounded in FactoryWave alongside the AURSAD and voraus-AD open-source datasets. Together, these provide a rigorous testbed for evaluating reasoning, causal understanding, and decision support over industrial signals.

1 Introduction

Modern industrial systems generate large volumes of multivariate time-series data from sensors, actuators, and controllers. In robotic manufacturing settings, these signals encode joint states, torques, forces, velocities, contact events, task phases, and fault indicators. Extracting actionable knowledge from such data is central to monitoring, diagnostics, anomaly detection, and decision support.

Traditional time-series models excel at narrow tasks such as prediction, classification, and anomaly detection Zhou et al. [2021], Su et al. [2019], Audibert et al. [2020]. However, these models are often specialized and difficult to interpret. They typically output labels or numeric predictions without explicit reasoning or higher-level explanations, and generalize poorly outside of the specific task they are trained on Chalapathy and Chawla [2019], Lavin and Ahmad [2015], Zeng et al. [2023], which limits their direct utility for automated engineering decision-making.

Large language models, in contrast, exhibit strong general reasoning abilities and can produce structured explanations in technical contexts Wei et al. [2022], Yao et al. [2023b]. They can integrate textual information, follow logical chains, and generate interpretable outputs. Yet, when applied directly to dense numerical time series, general LLMs still underperform without adaptation Gruver et al. [2023], Jin et al. [2024]. In contrast, specialized transformer-based architectures have become increasingly effective for time-series forecasting and representation learning Vaswani et al. [2017], Zhou et al. [2021], Wu et al. [2021], Zhou et al. [2022], Nie et al. [2023], Ansari et al. [2024], Das et al. [2024].

A promising paradigm is to build LLM agents equipped with specialized tools, including time-series models and signal-processing modules Schick et al. [2023], Yao et al. [2023a], Qin et al. [2023].

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Track on Evaluations and Datasets.

These agents can combine language reasoning with domain-specific computation. Nevertheless, evaluating whether such systems truly reason about machine behavior rather than pattern-matching on surface textual cues remains challenging. By machine-behavior understanding we mean the ability to (i) interpret the current operational state from raw multivariate signals, (ii) predict the effect of an intervention on the system, (iii) reason counterfactually about what would have happened under alternative histories, and (iv) recommend remedial actions grounded in physical and engineering constraints. To the best of our knowledge, how well current LLMs perform across these four competences on dense industrial telemetry has not yet been systematically measured. General reasoning benchmarks such as MMLU, BIG-bench, and MMMU Hendrycks et al. [2021], Srivastava et al. [2023], Yue et al. [2024] target textual, code, or multimodal understanding and are structurally unable to probe machine behavior; benchmarks closer in spirit, such as TimeSeriesExam, ChatTS, EngineMT-QA, TSAQA, Time-MQA, MTBench, and QuAnTS (Table 1), evaluate aspects of time-series Q&A but either restrict themselves to synthetic or univariate data, omit counterfactual reasoning, or do not involve a robotic system under closed-loop control, leaving a gap in rigorous evaluation of machine-behavior reasoning over industrial telemetry.

To address this gap, we propose FactoryBench, a benchmark designed to evaluate machine-behavior reasoning in time series models and LLMs. FactoryBench is grounded in FactoryWave, a dense multivariate time-series dataset collected from one-armed robotic systems spanning both collaborative and industrial manufacturing platforms, with synchronized setpoint, context, feedback, and effort signals recorded at 83 and 125 Hz (KUKA KR10 and UR3 respectively). Industrial-robot data of this kind is rare in public benchmarks: these systems are almost exclusively deployed in production environments with safety enclosures that restrict access, proprietary controllers that expose limited telemetry, and recordings that typically fall under plant-level confidentiality agreements. As a result, existing robotic datasets are dominated by cobot platforms, and to our knowledge FactoryWave is the first extensive anomaly dataset to (partly) cover an industrial robot, explicitly aimed at bridging this cobot-to-industrial-machine gap.

We also introduce a scalable question-generation framework composed of 20 structured templates spanning a wide range of reasoning types and applicable to general machine data flows. These templates enable systematic construction of question-answering tasks that probe state understanding, intervention reasoning, counterfactual analysis, and decision-making in machine contexts. Using this framework and the FactoryWave dataset, we build FactoryBench, a large-scale Q&A dataset specifically designed to evaluate time-series understanding and engineering reasoning in LLM agents.

By bridging the gap between general language reasoning and specialized time-series modeling, FactoryBench provides a principled foundation for studying machine-behavior reasoning in industrial environments.

2 Related work

Recent advances in LLM reasoning include prompting and deliberative strategies (e.g., Chain-of-Thought and Tree-of-Thought) that improve multi-step performance Wei et al. [2022], Yao et al. [2023b], alongside tool-augmented paradigms such as Toolformer and ReAct that enable grounded computation through external modules Schick et al. [2023], Yao et al. [2023a], Qin et al. [2023]. These ideas motivate industrial agent design where symbolic reasoning must be combined with numerical workflows.

Time-series modeling has advanced through transformer-based architectures (Informer, Autoformer, FEDformer, PatchTST) Zhou et al. [2021], Wu et al. [2021], Zhou et al. [2022], Nie et al. [2023], strong alternative baselines such as TFT and N-BEATS Lim et al. [2021], Oreshkin et al. [2020], and critical comparisons against simpler linear models Zeng et al. [2023]. Foundation-model directions (e.g., Chronos) further explore transfer and zero/few-shot adaptation Ansari et al. [2024], Das et al. [2024]. In parallel, anomaly and reliability monitoring literature includes OmniAnomaly, USAD, and Deep SVDD Su et al. [2019], Audibert et al. [2020], Ruff et al. [2018], with surveys and benchmarks highlighting persistent gaps in interpretability and root-cause actionability Chalapathy and Chawla [2019], Lavin and Ahmad [2015].

Causal frameworks provide formal tools for interventions and counterfactuals Pearl [2009], Peters et al. [2017], Rubin [1974], while temporal methods such as Granger causality and modern nonlinear causal discovery support directional reasoning in time series Granger [1969], Runge et al. [2019].

Existing broad benchmarks (MMLU, BIG-bench, MMMU) Hendrycks et al. [2021], Srivastava et al. [2023], Yue et al. [2024] and classical time-series resources Laptev et al. [2015], Dau et al. [2019], Wen et al. [2022] do not directly evaluate machine-centered reasoning over industrial telemetry. FactoryBench targets this gap by jointly testing temporal interpretation, causal reasoning, and engineering decision support. Table 1 positions FactoryBench against closely related Q&A and time-series benchmarks along eight axes. The column labels refer to concepts introduced later in the paper: Counterfactual denotes questions that reason about alternative histories (Level 3, Section 3.1); Free-Form denotes open-ended natural-language answers, scored deterministically against a reference rubric (Appendix A); and Novel Dense TS indicates whether the benchmark contributes a new, high-frequency multivariate dataset rather than relying exclusively on public sources.

Table 1: Comparison of FactoryBench with existing benchmarks. "Counterfactual" = questions reasoning over alternative histories; "Free-Form" = open-ended answers judged by LLMs; "Novel Dense TS" = contributes a new multivariate high-frequency dataset.

Benchmark	Machine/Robot	Multivariate TS	Size	Counterfactual	Ranking	Numerical	Free-Form	Novel Dense TS
TimeSeriesExam	×	×	∼700	×	✓	×	×	×
ChatTS	×	✓	∼134k	×	✓	✓	×	×
EngineMT-QA	✓	✓	∼110k	×	✓	✓	✓	×
TSAQA	Partial	×	∼210k	×	✓	×	×	×
Time-MQA	×	✓	∼200k	×	✓	✓	✓	×
MTBench	×	✓	?	×	✓	✓	✓	×
QuAnTS	×	✓	∼150k	×	✓	✓	✓	×
CLEVR	×	×	∼1M	×	×	✓	×	×
CLEVRER	×	×	∼305k	✓	×	✓	✓	×
FactoryBench (Ours)	✓	✓	**TBD**	✓	✓	✓	✓	✓

3 FactoryBench framework

3.1 Four levels of machine understanding

Table 2: FactoryBench levels, what they test, example questions, and expected answer formats.

Level	Example question	Type	What it tests	Answer format
1	"We want to isolate the lifting phase in the robot's time series. Assuming a fixed window length of 15 timesteps, at which timestamp should the window begin?"	State	Can the model understand what the machine is doing and how it is behaving?	Tensor
2	"A collision with a foam cube occurs at $T{=}850$ ms. Rank signal segments (A–D) in the order you would expect them to appear after the event."	Intervention	Can the model reason about how the machine will behave in the face of an event?	Ranking
3	"Had a payload misconfiguration occurred at $T{=}200$ ms, what would the target torque on joint 2 at $T{+}50$ ms have been in this counterfactual case?"	Counterfactual	Can the model reason accurately about theoretical scenarios?	Scalar
4	"Given the sensor stream below, does the machine show signs of anomalous behavior? If yes, identify the root cause and the steps to fix it."	Decision	Can the model make informed decisions about the machine?	Free-form

Our four-tier hierarchy is explicitly built on top of Pearl's ladder of causation association, intervention, and counterfactual reasoning which has become the organizing principle for causal inference and is increasingly adopted as an evaluation axis for machine-learning systems Pearl [2009], Peters et al.

[2017], Jin et al. [2023], Pawlowski et al. [2020], Peyrard and West [2020]. We extend the three causal rungs with a fourth decision-making rung that reflects how industrial robotic platforms are actually operated: after interpreting state and causal relationships, operators must select and execute corrective procedures, typically prescribed by vendor manuals (e.g., Universal Robots error-code handbooks). This final rung aligns with the diagnostic-then-act loop that is standard in fault-tolerant control. Grounding the hierarchy in these established principles ensures that each level probes a qualitatively distinct reasoning skill and that failure modes are interpretable in terms of the underlying causal or decision-theoretic primitive.

To systematically evaluate machine-behavior reasoning, we organize question-answering tasks according to this four-tier hierarchy, each probing distinct reasoning capabilities:

Level 1: State. This level assesses the agent's ability to interpret the current state of the machine under normal conditions, including identification of operational modes, prediction and recognition of sensor patterns. Questions at this level require accurate extraction and interpretation of time series features.

Level 2: Intervention. At this level, the agent must reason about the consequences of interventions or events occurring at the present timestep that might perturb the distribution of machine states. Tasks include predicting the immediate impact of control actions, diagnosing faults as they arise, and understanding causal relationships in real time for both normal and anomalous episodes.

Level 3: Counterfactual. This level evaluates the agent's ability to reason about hypothetical scenarios, such as the effect of an event or intervention at a previous timestep. Questions require the agent to simulate alternative histories and assess how outcomes would differ under counterfactual conditions, while still considering the history they know.

Level 4: Decision Making. The highest level evaluates the agent's ability to act on its understanding of the system. Given a recorded episode and a task description, the agent must produce a sequence of actions or recommendations to answer that prompt. In FactoryBench, this targets troubleshooting and optimization, and combines the skills exercised at the previous three levels: reading the system state, reasoning about causes, and weighing alternative interventions. It reflects what we expect from an LLM acting as an engineering assistant on the factory floor.

By structuring Q&A tasks along these four levels, FactoryBench enables rigorous and granular assessment of machine-behavior reasoning, from basic state recognition to advanced decision support. Figure 1 instantiates Pearl's three causal rungs (L1–L3) on robotic time-series data and adds the L4 decision-making layer that FactoryBench introduces on top. Each question is emitted in one of five answer formats (single-select MCQ, multi-select MCQ, ranking, tensor, free-form), all of which support deterministic scoring; the per-format rubrics and tolerance bands are detailed in Appendix A.

3.2 Answer formats

FactoryBench uses four answer formats, chosen to balance evaluation rigor with scalability. Each format supports deterministic scoring (with the exception of free form), and has been chosen in order to make questions very easily checkable, but also hard to answer in the absence of correct reasoning. The procedure that turns these per-format rubrics into a per-item score — including how non-canonical model outputs are recovered without delegating the verdict to an LLM — is described in Section 5.4, with the full three-step implementation specified in Appendix D.

3.3 Time series data and causal schema (SCE)

So that the dataset structure itself encodes causality, all time-series signals are organized into three causal groups: **Setpoint** (the controller's commanded target), **Context** (physical and configured conditions, split into static episode metadata and dynamic time-series channels), and **Effort+Feedback** (the machine's measured response). Under healthy operation the response is a known function of setpoint and context; faults manifest as structured residuals from that baseline, giving a single operational fault definition that applies across machines and tasks. Full per-group channel mappings, the formal residual definition, and per-robot calibration details are deferred to Appendix K.

The data conforms to this schema across two source types:

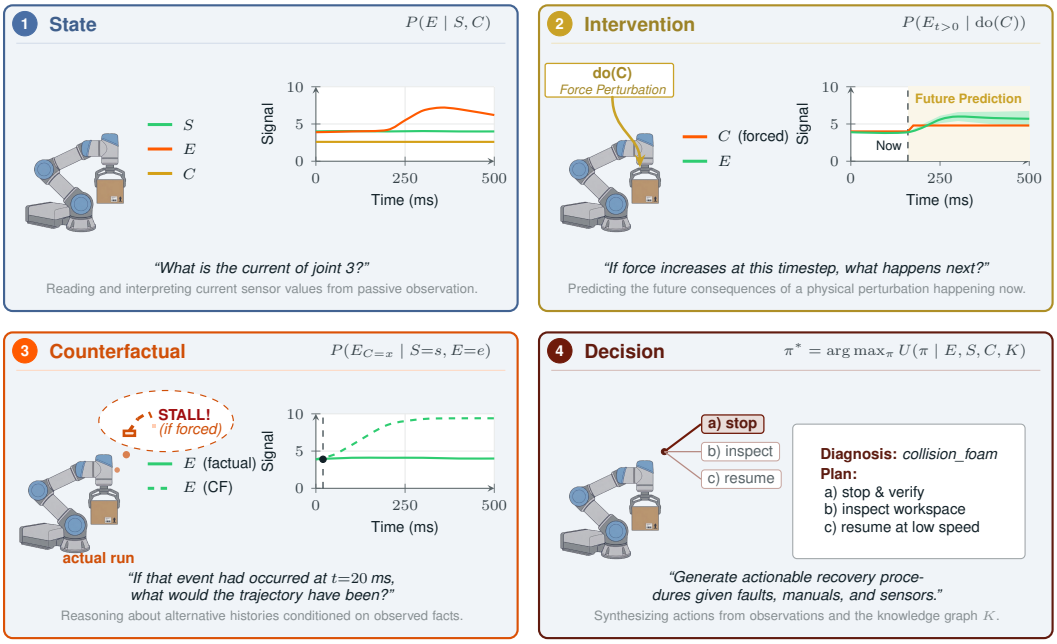

Figure 1: Pearl's three rungs of causal reasoning (L1–L3), instantiated on robotic time-series, plus the FactoryBench L4 decision-making layer. Signals follow the SCE schema: S is the *setpoint* (the controller's commanded target), E is the *effort* (the machine's measured response, e.g., motor current or torque), and C is the *context* (physical and configured operating conditions, e.g., payload mass). **L1 (State)** captures correlations from passive observation; **L2 (Intervention)** predicts the future effect of a present physical perturbation; **L3 (Counterfactual)** reasons over alternative histories conditioned on what was observed; **L4 (Decision)** synthesizes a recovery procedure by combining the observed signals with manufacturer manuals and the FactoryBench knowledge graph K.

Table 3: Answer formats used in FactoryBench and their evaluation protocols.

Format	**Description**	**Evaluation**
Multi-Select MC	Four independent statements (A–D) about the outcome of an event or intervention; the model selects any subset.	Score of 1 for exact match, 0.5 for 1 mistake, 0 otherwise.
Ranking	Four time-series segments or outcomes (A–D) ordered by a specified criterion (e.g., severity, magnitude). Answer is a permutation string (e.g., `DABC`).	Exact-match rate and Kendall's τ rank correlation against the ground-truth ordering.
Tensor	One or more scalar values (e.g., expected sensor reading after an intervention) with no multiple-choice scaffolding. Answer is a list of floats.	Per-scalar MAPE and threshold-based accuracy ($\pm k\%$ of ground truth), with partial credit for each correct scalar.
Free-Form	Open-ended natural language response (diagnosis, action sequence, or causal explanation).	LLM-as-judge voting: three frontier LLMs each cast **Wrong** (0), **Neutral** (0.5), or **Correct** (1); final score is the majority vote, ties $\rightarrow$ 0.5

- **FactoryWave:** A custom dataset generated from one-arm robotic platforms executing canonical industrial tasks (e.g., pick-and-place) under varied conditions and systematically injected anomalies.
- **Open Source Datasets:** Adapted datasets such as AURSAD and voraus-AD, offering diverse industrial scenarios and preprocessed to conform to the unified episode structure.

The datasets integrated into FactoryBench are summarized in Table 4; all provide the full setpoint–context–effort triplet at 83 Hz or higher and have been (re)labelled to conform to our unified episode schema.

Table 4: Datasets incorporated into FactoryBench. FactoryWave is collected and released with this work; the other two are open-source datasets adapted to our schema. Tasks: PnP = pick-and-place, Scr = screwing, PiH = peg-in-hole.

Dataset	Robot	Episodes	Frequency	Tasks	Anomalies
AURSAD	UR3e	4094	100 Hz	Scr	4
voraus-AD	Yu-Cobot	2122	100/500 Hz	PnP	12
FactoryWave (ours)	UR3, KUKA KR10	7142	125/83 Hz	PnP, Scr, PiH	27

4 Reliable Q&A generation

4.1 At scale generation via extensive labelling

Scalable ground-truth generation is the central challenge of any Q&A benchmark grounded in raw data. FactoryBench addresses this by coupling a structured labelling ontology with a context-free grammar (CFG)-style template system, designed by robotics experts and PhD researchers. Each template contains variable slots filled at generation time from one of two sources: label information read directly from the episode data (signal names, timestamps, anomaly labels, root causes), or a predefined sampling distribution attached to that slot (prediction horizons, numerical thresholds, curated vocabularies for discrete options). The discrete vocabularies are seeded from the label sets of AURSAD and voraus-AD, extended to cover FactoryWave-specific cases, and released in full so every instantiation is inspectable and reproducible. The context time series used to fill the variables is sampled uniformly by data source (open source vs FactoryWave), then dataset, experiment, length within controlled bounds, and finally placement, yielding a combinatorial expansion from a small set of carefully designed templates into a large, diverse question pool.

Each of the 20 templates is manually authored to probe one specific machine-understanding level while remaining general enough to admit a wide range of instantiations across episode segments, signal names, event descriptions, timestamps, and predicted values. Answer options for single- and multi-select questions are drawn from a shared pool of pre-defined verifiable statements, each paired with a rule that can be evaluated deterministically against the time series given the densely labelled data, so ground truth is never imputed or inferred but computed directly from the underlying labels. This is also our primary defence against the natural concern that template-generated benchmarks can be solved by learning template surface structure rather than the underlying reasoning: every template admits a combinatorially large instantiation space over signal names, timestamps, prediction horizons, payload conditions, and injected-fault types, so two questions sharing the same template rarely overlap in more than a fraction of their variable slots, and the correct answer is never recoverable from the question text alone since it is always determined by the paired time series, which varies across instantiations.

4.2 Density of FactoryWave

FactoryWave is collected from two physical robots (UR3 and the KUKA KR10) executing up to three canonical industrial tasks each: pick-and-place, screwing (UR3 only), and peg-in-hole. Each complete task cycle constitutes one episode, recorded at 125 and 83 Hz across sensor channels covering joint setpoints, feedback, torques, speeds, estimated contact forces, TCP pose, gripper state, and task-phase labels.

find number

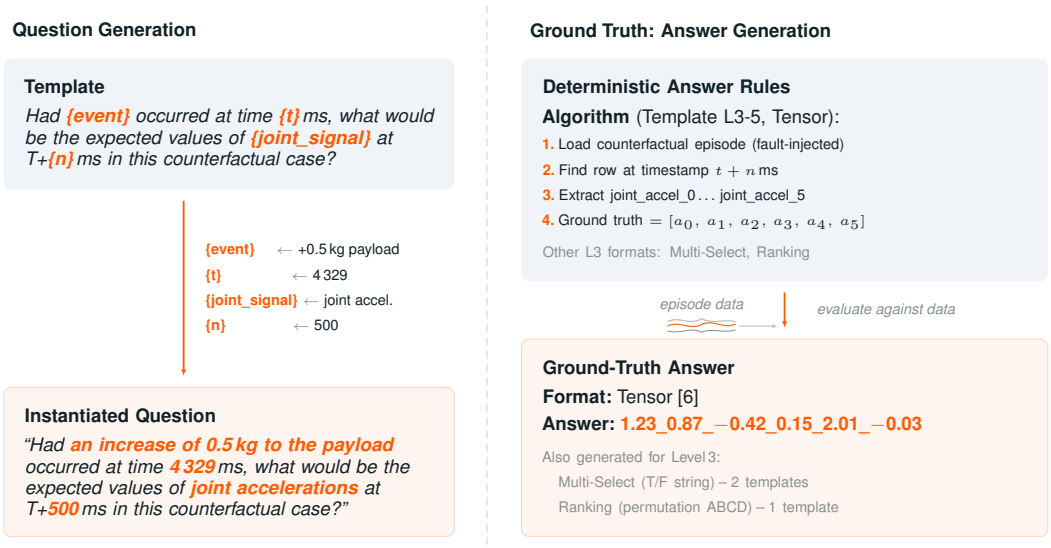

Figure 2: Question-and-answer generation pipeline (Level 3 Template 5 shown; full inventory in Table 8 and Appendix G). **Left:** Typed placeholder slots are bound to episode-specific values. **Right:** Ground-truth answers are computed deterministically from paired episode data via template-specific rules.

Episodes are organized into three experiment types. **Normal** episodes capture nominal operation across three payload levels (light, medium, heavy for pick-and-place). **Fault** episodes inject one of 27 fault types (spanning gripper failures, payload misconfigurations, collisions, and peg-in-hole-specific anomalies), each recorded with fault-specific metadata and, where applicable, a precise injection timestep. **Counterfactual** episodes provide ground truth for Level 3 causal reasoning by approximating the do-operation on physical hardware, via the protocol described next.

Counterfactual generation approximates Pearl's do-operator on physical hardware, which cannot reproduce a bit-identical pre-injection state. For each baseline we re-execute the task 3–5 times with every controllable condition held fixed (object pose, payload, controller configuration, exact scripted trajectory) and inject the target fault at a fixed timestep $t^\star$. We then keep the run whose pre-injection segment minimises KL divergence against the baseline, and use that episode as an approximate groundtruth for hypothetical divergence of baseline.

Table 5 summarises the resulting distribution of episodes across conditions and tasks. **Counterfactual** entries count CF *groups*: each group is stored as a single paired episode bundling the baseline run with its retained fault counterpart.

Table 5: Distribution of FactoryWave episodes by condition and task. Counterfactual entries count CF groups (one baseline plus one retained fault run per group).

Condition	Pick-and-Place	Screwing	Peg-in-Hole	Total
Normal	1053	230	322	1605
Fault	3238	850	1200	5288
Counterfactual (groups)	109	40	100	249
Total	**4400**	**1120**	**1622**	**7142**

4.3 Knowledge graph and protocol extraction

Reliable ground truth for Level 4 troubleshooting questions requires more than episode labels: it demands machine-specific recovery protocols grounded in manufacturer documentation. For each robot in FactoryBench we parse the manufacturer's runtime error documentation into a structured mapping from error codes to error names, descriptions, and recommended recovery steps, capturing what the controller reports when a fault occurs and what an operator should do to resolve it. We then

map every anomaly type studied in FactoryBench to the most likely corresponding manufacturer error code, with PhD-level robotics experts reasoning about the physical cause of each anomaly and identifying which runtime error it would most plausibly trigger on the target robot. Where a physical fault injection (such as gripper misactivation, peg misalignment, or external collision) has no well-defined controller error, the same experts author the recovery protocol directly using the same structure for consistency.

The complete mapping (error-derived and expert-authored protocols alike) is ingested into a knowledge graph alongside robot specifications, task definitions, and signal schema. At question generation time, the pipeline queries this graph to automatically assemble the relevant protocol context and inject it as ground truth into Level 4 troubleshooting Q&A pairs, without requiring per-question human review.

5 Experiments

5.1 Zero-shot evaluation

We evaluate a cross-vendor panel of frontier LLMs available at submission time (April 2026): GPT-5.1 OpenAI [2025] and Claude Sonnet 4.6 Anthropic [2025] on the closed-weight side, DeepSeek 3.2 DeepSeek-AI [2025] and Mistral-Large-3 Mistral AI [2025] on the open-weight side. Qwen3.5-4B Qwen Team, Alibaba [2026] is reserved for the inter-robot generalisation experiment in Section 5.2 and is therefore not included in the zero-shot table. For calibration we also report a non-LLM baseline that dispatches per item by answer format: linear regression on the target signal for the scalar and tensor forecasting templates, and seeded uniform random over the option set for single-select MCQ, multi-select T/F, ranking, and the phase-window numeric templates. Free-form Level 4 templates are out of scope for this baseline (no traditional method maps cleanly to "produce a remediation protocol") and are excluded from the baseline column.

Table 7 reports measured mean rubric scores for the panel and the baseline.

Table 6: Measured mean rubric scores (%) on FactoryBench's four reasoning levels. Each cell is the mean over $n = 100$ Q&A pairs unless noted; All scoring is computed per the per-format rules in Appendix A.

Method	L1	L2	L3	L4
Simple baseline	21.8%	29.3%	29.1%	n/a
Claude Sonnet 4.6	33.5%	44.0%	48.2%	5.2%
DeepSeek 3.2	23.0%	25.0%	30.1%	5.1% (n=73)
GPT-5.1	32.0%	35.1%	36.0%	19.5%
Mistral-Large-3	32.2%	32.9%	46.2%	6.9% (n=59)

Two patterns stand out. On L1–L3 the LLM panel separates cleanly from the baseline, with Claude Sonnet leading on L2 and L3 (44.0% and 48.2%) and Mistral-Large-3 close behind on L3 (46.2%); DeepSeek is the exception, sitting at or below the baseline on L1 and L2 and only barely above it on L3, which suggests it does not exploit the time-series context as effectively as the other panel members. On L4 every model collapses into the 5–20% range despite the deterministic rubric only requiring identification of the root cause and remediation protocol; GPT-5.1 leads at 19.5% but that mass comes largely from correctly flagging nominal episodes as fault-free rather than from producing correct remediation procedures. The L1–L3 gap and the L4 collapse together indicate that current frontier models read industrial telemetry well enough to handle state and intervention questions but still fall short of the engineering-decision rung of the hierarchy.

5.2 Industrial generalization

Beyond the zero-shot panel, we run a targeted finetuning experiment to probe whether machine-behavior understanding can be *learned* from FactoryBench rather than only inherited from a model's pretraining priors, and whether what is learned transfers across machine families. We finetune Qwen3.5-4B on the subset of FactoryBench Q&A pairs grounded in cobot episodes: the UR3 from FactoryWave, the UR3e from AURSAD, and the Yu-Cobot from voraus-AD. The training split spans

all four reasoning levels and every answer format, but contains no episodes from industrial-grade hardware.

We then evaluate the finetuned checkpoint on a held-out test split composed exclusively of Q&A pairs grounded in industrial-robot episodes (the KUKA KR10 portion of FactoryWave and the heavier-payload industrial platforms in the simulation pool), with no industrial-robot data seen during training. This split tests a generalisation claim that is operationally important for industrial AI: cobots are cheap and safe to instrument, vary, and inject faults into, while production industrial robots, with their high payloads, high speeds, and stricter safety-stop interlocks, are not. A positive transfer result, where the finetuned model's score on the industrial test split is close to or above the zero-shot panel, would indicate that the diagnostic and counterfactual reasoning skills FactoryBench measures can be acquired cheaply on collaborative platforms and deployed on production hardware. A negative result would localise where machine-behavior reasoning is platform-specific and quantify the cobot-to-industrial gap, which is itself a useful artefact for future benchmarks and sim-to-real training pipelines.

5.3 Sim2real gap analysis on counterfactual pick-and-place

Level 3 questions require comparing what *did* happen against what *would have* happened under a different intervention, holding the rest of the world fixed. Physical executions cannot reproduce identical initial conditions, micro-disturbances, or contact dynamics, so a physical "counterfactual pair" is only an approximation of the intended do-operation; simulation, by contrast, can deterministically fork its state and yield bit-identical pre-intervention trajectories. Simulation buys exact counterfactual fidelity at the cost of realism (contact, noise, latency, and manufacturing variability all drift from the physical system) so we treat the two sources as complementary.

In FactoryWave we approximate the do-operation by fixing initial conditions (object, pose, payload, controller config), recording one clean baseline, then 3–5 fault runs with the fault event injected at the same fixed timestep, and retaining the fault run with the lowest pre-injection KL divergence against the baseline as that baseline's counterfactual partner (full protocol in Appendix F). In IsaacSim we reproduce the same pick-and-place task with a URDF-matched UR3 and matching workspace; for each episode we run the baseline to completion, then re-run from a saved simulator state at the injection timestep with the fault applicator enabled, so the two branches are bit-identical pre-intervention by construction and require no KL selection. This lets us generate arbitrarily many counterfactual pairs at negligible cost, including for faults that are too disruptive or unsafe to stage repeatedly on hardware.

Fill in concrete numbers, plots, and per-template gap tables once the IsaacSim pipeline is frozen; see Appendix H.

5.4 Evaluation of LLM answers

Free-form answers are LLM-judged by construction (Section 3.2); the other five answer formats admit a canonical answer string and therefore a deterministic check. Every L1–L3 answer is graded by a strict three-step cascade (Figure 3) that uses an LLM only as a transcriber when the model's output cannot be parsed, never as the source of the score.

Scoring cascade: a three-step verifier. Whether a model's answer is correct is decided by a strict three-step procedure that keeps every L1–L3 score *deterministic by construction* while still tolerating chatty or off-format model outputs:

1. **Step 1 — Deterministic parsing.** The raw model response is fed to a per-format parser keyed by `answer_format`: single-select MCQ (final letter in $\{A, B, C, D\}$), multi-select MCQ (n-character T/F string), ranking (permutation over the option alphabet), numerical (scalar with per-question tolerance), or tensor (underscore-separated floats). This handles the overwhelming majority of well-behaved responses.

2. **Step 2 — Judge extraction.** When Step 1 cannot recover an answer (e.g. the model embeds it in long prose), a single LLM-as-judge call is made with a per-format extraction prompt. The judge's job is to recover the model's intended answer in the canonical string format ($A/B/C/D$, the n-character T/F string, the permutation, or the underscore-separated tensor).

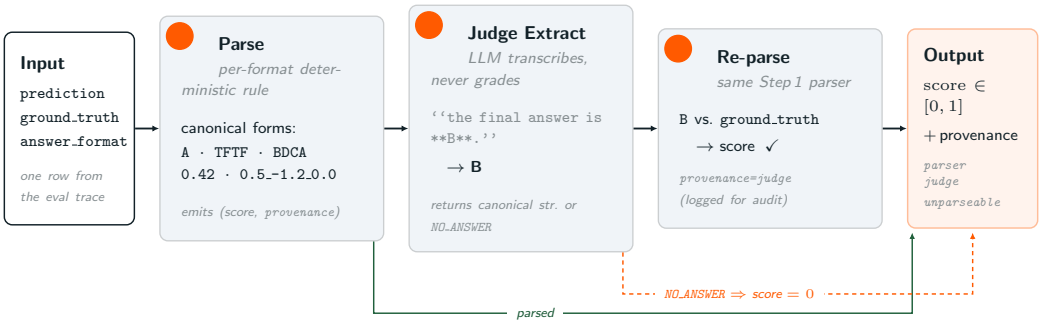

Figure 3: Three-step scoring cascade for FactoryBench's deterministic answer formats (single-/multi-select MCQ, ranking, numerical, tensor; cf. Section 3.2). **Step 1** runs the per-format parser on the raw response; when it succeeds, the cascade exits with a deterministic score and `provenance=parser` (green return path). **Step 2** is invoked only on unparseable responses: an LLM is asked to *transcribe* (not grade) the model's intended answer into the canonical string, returning either that string or `NO_ANSWER` (orange dashed path forces score $= 0$). **Step 3** feeds the extracted string back through the same Step 1 parser, so the verdict is still produced by the deterministic rule even when the judge mediated extraction (`provenance=judge`, logged for auditability). Free-form (Level 4) answers bypass this cascade and are graded by the LLM-judge rubric of Section 3.2.

3. **Step 3 — Re-parse and verify.** The extracted string from Step 2 is fed back to the same deterministic parser of Step 1, which produces the final score against the ground truth answer.

Every L1–L3 score is reducible to the deterministic parser, the judge is invoked only on the long tail of unparseable outputs, and only free-form scoring depends on the judge's rubric judgement. Free-form Level 4 answers are out of this cascade by design: they are scored directly by the same judge under the $\{0, 0.5, 1\}$ rubric of Section 3.2. Full implementation detail (parser logic per format, tolerance bands, judge prompts, escalation policy) is given in Appendix D.

5.5 Results

Coral: we should update this section after the final run, incluiding the final model list

Table 7 reports measured mean rubric scores for our cross-vendor LLM panel and the simple non-LLM baseline. The baseline dispatches per template type (linear regression on the target signal for tensor-prediction templates, and seeded uniform random over the option set for MC, multi-select and ranking templates) and is therefore deliberately weak: it cannot read the question or use the rest of the context. Level 4 is free-form and excluded from the baseline column because no traditional method maps cleanly to "produce a remediation protocol".

Table 7: Measured mean rubric scores (%) on FactoryBench's four reasoning levels, comparing the simple non-LLM baseline against four frontier LLMs. Each cell reports the mean over $n = 100$ Q&A pairs (gpt-5.1 L4 has $n = 85$ due to API failures). Levels 1–3 use the deterministic per-format scorer described in Section 3.2; L4 is free-form and uses the rubric described above. The baseline is intentionally answer-format-agnostic: it never beats a model that genuinely reads the time series, so any cell where a model underperforms it is informative.

Method	L1	L2	L3	L4
Simple baseline	20.5%	19.5%	10.3%	n/a
Claude Haiku 4.5	19.0%	25.5%	6.3%	0.5%
DeepSeek V3.1	16.5%	23.7%	24.7%	3.5%
gpt-5.1	24.0%	28.7%	22.0%	7.6%
Mistral-Large-3	22.0%	22.7%	26.7%	0.0%

Two facts in Table 7 stand out. First, every model is at or below the random/regression baseline on Level 1, and several (Claude Haiku and DeepSeek) are also dominated on L2; on Level 3 the LLMs pull ahead by 12–16 points but Claude Haiku still loses to the baseline. Second, Level 4 free-form rubric scores are extremely low across the board (0–7.6%), with gpt-5.1 the only model recording any non-trivial mass: not because it produces correct remediation protocols, but because it correctly identifies several "no anomaly" episodes verbatim.

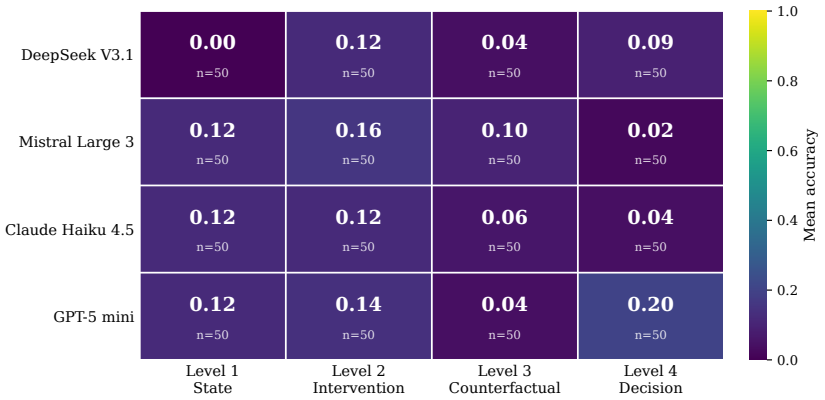

Figure 4: Measured accuracy of four frontier models evaluated via Microsoft Foundry across FactoryBench's four reasoning levels. Each cell reports the mean accuracy over n evaluated Q&A pairs. Levels 1–3 and Level 4 ranking templates use deterministic scoring; Level 4 free-form templates (troubleshooting and optimization) are scored against the reference root cause and remediation protocol under a $\{0, 0.5, 1\}$ rubric.

PP: I find it pretty remarkable that Deepseek scores a flat 0.00 in Figure 4, even though it did not fail completely in Table 7. I think this should be discussed. It appears though as if Figure 4 is not referenced in the text.

6 Discussion

6.1 Why this benchmark matters

For researchers: First rigorous evaluation of machine understanding across reasoning levels.

For practitioners: Quantifies which AI capabilities are ready for deployment as AI engineers.

For the field: Defines "machine understanding" operationally via Q&A.

6.2 Modular and extensible by design

FactoryBench is built to grow. The Q&A generator decouples reasoning templates from the underlying robot, task, and signal schema: every template is parameterised over generic primitives (signal names, phase indices, anomaly labels, fault categories) rather than hard-coded to a particular machine or task, so adding a new platform only requires populating the corresponding entries in the labelling ontology and the per-robot signal mapping. Once those are in place every existing template instantiates against the new platform automatically, and the answer-format scorers, the knowledge graph, and the LLM-as-judge prompts all carry over. As the project evolves we plan to incorporate additional robots and machine families (industrial arms, mobile manipulators, CNC and additive-manufacturing platforms), additional tasks (assembly variants, polishing, inspection), additional gripper and end-effector types, and additional question templates targeting under-represented reasoning patterns. Each addition slots into the existing schema without breaking prior versions, and the versioned release track ensures that every reported number remains pinned to a fixed dataset state while the benchmark itself continues to grow.

6.3 Limitations

FactoryBench has two substantive limitations that future work must address rather than gloss over. First, Level 3 *counterfactual* ground truth in the physical lab is fundamentally approximate: two real runs cannot share an identical pre-injection state, so our KL-divergence-selected CF pair is only the closest empirical proxy for the do-operation, not the do-operation itself. L3 scores therefore conflate the model's counterfactual reasoning with the quality of that proxy, and simulation closes the gap only at the cost of physical realism. Second, the Q&A pairs are built on a simplified representation of how machine faults actually appear in production, since it is the first benchmark of its sort. Real industrial faults are typically compound (multiple co-occurring root causes), gradual (slow drift over shifts or weeks), and detected through aggregate KPIs rather than single-episode signals; ours are atomic, fast, and drawn from a closed catalogue of 27 physically injected mechanisms, which makes FactoryBench a measurement of in-distribution fault *recognition* rather than the fault *detection* problem operators actually face.

7 Conclusion

FactoryBench introduces a systematic approach to evaluating machine understanding via Q&A. By focusing deeply on one machine family and providing reliable answer generation at scale, we enable rigorous comparison of methods across four levels of reasoning from basic state identification to procedure generation grounded in technical manuals.

Our physical validation protocol closes the loop from benchmark to reality, addressing the fundamental question: can AI systems that excel at industrial Q&A actually help in the real world?

7.1 License

Some practical questions: How can people employ the benchmark? Where can it be found. Will it be maintained and expanded? Will incorrect labels be corrected? Is there a private dataset too? Are parts of the dataset designated for training and parts for testing?

FactoryBench and FactoryWave are released under a license and distributed via a public Hugging Face repository. The release is split into two repos: `Forgis/FactoryNet_Dataset` hosts the raw episode data and labels, while `Forgis/FactoryBench_QA_pairs` hosts the generated Q&A snapshots organised by level (`level_1/`, ..., `level_4/`) and by source dataset folder, so that any model can be evaluated against a fixed Q&A set without re-running the generator. The release also includes the generator source code, the full set of 20 question templates with their paraphrase banks, all LLM-as-judge prompts, and both the full benchmark and the FactoryBench-Lite subset (Section **??**). We provide a versioned release track (e.g., `v1.0`, `v1.1`) so that reported numbers always refer to a fixed benchmark state; corrections to labels or templates are published as minor-version updates with a public changelog, and the exact version used in any evaluation is stamped into every result file.

check license

References

Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, Jasper Zschiegner, Danielle C. Maddix, Hao Wang, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Yuyang Wang. Chronos: Learning the language of time series. *TMLR*, 2024. arXiv:2403.07815.

Anthropic. Claude Haiku 4.5: Model card, 2025. Anthropic model card.

Julien Audibert et al. Usad: Unsupervised anomaly detection on multivariate time series. *KDD*, 2020.

Raghavendra Chalapathy and Sanjay Chawla. Deep learning for anomaly detection: A survey. *arXiv preprint*, 2019.

Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. A decoder-only foundation model for time-series forecasting. In *ICML*, 2024. arXiv:2310.10688.

Hoang Anh Dau et al. The ucr time series classification archive. *IEEE/CAA Journal of Automatica Sinica*, 2019.

DeepSeek-AI. DeepSeek-V3.1 technical report, 2025. DeepSeek-AI technical report.

Clive W. J. Granger. Investigating causal relations by econometric models and cross-spectral methods. *Econometrica*, 1969.

Nate Gruver, Marc Finzi, Shikai Qiu, and Andrew Gordon Wilson. Large language models are zero-shot time series forecasters. *NeurIPS*, 2023.

Dan Hendrycks et al. Measuring massive multitask language understanding (mmlu). *ICLR*, 2021.

Ming Jin et al. Time-llm: Time series forecasting by reprogramming large language models. *ICLR*, 2024.

Zhijing Jin, Yuen Chen, Felix Leeb, Luigi Gresele, Ojasv Kamal, Zhiheng Lyu, Kevin Blin, Fernando Gonzalez Adauto, Max Kleiman-Weiner, Mrinmaya Sachan, and Bernhard Schölkopf. CLadder: A benchmark to assess causal reasoning capabilities of language models. In *NeurIPS*, 2023.

Nikolay Laptev, Saeed Amizadeh, and Ian Flint. Generic and scalable framework for automated time-series anomaly detection. *KDD*, 2015.

Alexander Lavin and Subutai Ahmad. Evaluating real-time anomaly detection algorithms – the numenta anomaly benchmark. *IEEE ICMLA*, 2015.

Błażej Leporowski, Daniella Tola, Christian Hansen, and Alexandros Iosifidis. AURSAD: Universal robot screwdriving anomaly detection dataset. *arXiv preprint arXiv:2202.03211*, 2022.

Bryan Lim et al. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *IJF*, 2021.

Mistral AI. Mistral Large 3: Model card, 2025. Mistral AI model card.

Yuqi Nie et al. A time series is worth 64 words: Long-term forecasting with transformers (patchtst). *ICLR*, 2023.

OpenAI. GPT-5-mini: System card and technical report, 2025. OpenAI technical report.

Boris N. Oreshkin et al. N-beats: Neural basis expansion analysis for interpretable time series forecasting. *ICLR*, 2020.

Nick Pawlowski, Daniel C. Castro, and Ben Glocker. Deep structural causal models for tractable counterfactual inference. In *NeurIPS*, 2020.

Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.

Jonas Peters, Dominik Janzing, and Bernhard Schölkopf. *Elements of Causal Inference*. MIT Press, 2017.

Maxime Peyrard and Robert West. A ladder of causal distances. *arXiv preprint arXiv:2005.02480*, 2020.

Yujia Qin et al. Tool learning with foundation models. *arXiv preprint*, 2023.

Qwen Team, Alibaba. Qwen-3.5 technical report, 2026. Alibaba technical report.

Donald B. Rubin. Estimating causal effects of treatments in randomized and nonrandomized studies. *Journal of Educational Psychology*, 1974.

Lukas Ruff et al. Deep one-class classification. *ICML*, 2018.

Jakob Runge et al. Detecting and quantifying causal associations in large nonlinear time series datasets. *Science Advances*, 2019.

Timo Schick et al. Toolformer: Language models can teach themselves to use tools. *NeurIPS*, 2023.

Aarohi Srivastava et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models (big-bench). *TMLR*, 2023.

Ya Su, Youjian Zhao, Chenhao Niu, Rong Liu, Wei Sun, and Dan Pei. Robust anomaly detection for multivariate time series through stochastic recurrent neural network. In *KDD*, 2019.

Ashish Vaswani et al. Attention is all you need. *NeurIPS*, 2017.

Jason Wei et al. Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*, 2022.

Qingsong Wen et al. Transformers in time series: A survey. *IJCAI*, 2022.

Haixu Wu et al. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *NeurIPS*, 2021.

Shunyu Yao et al. React: Synergizing reasoning and acting in language models. *ICLR*, 2023a.

Shunyu Yao et al. Tree of thoughts: Deliberate problem solving with large language models. *NeurIPS*, 2023b.

Xiang Yue et al. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. *CVPR*, 2024.

Ailing Zeng et al. Are transformers effective for time series forecasting? *AAAI*, 2023.

Haoyi Zhou et al. Informer: Beyond efficient transformer for long sequence time-series forecasting. *AAAI*, 2021.

Tian Zhou et al. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. *ICML*, 2022.

Do all the references actually exist? If they do not, this is typically seen as a proof that we had this paper AI generated. It will then likely be desk-rejected.
Before submission, double-check every reference in `references.bib`:
A) Verify authors, title, venue, and year against the original source. B) Check for fabricated or near-duplicate entries. C) When citing arXiv papers, include the identifier in the citation, e.g. arXiv:2201.12740. D) Capitalization: the references currently write everything in lowercase, but some abbreviations are chosen by their authors to capitalize specific words. Follow the authors' preferred formatting.
Specific entries flagged so far:
- Could not find: Abhimanyu Das et al. "Time series foundation models and forecasting". - Slightly different name: Ya Su et al. "Robust anomaly detection for multivariate time series through stochastic recurrent neural network (OmniAnomaly)". - Wrong author: Gerald Woo et al. "Chronos: Learning the language of time series", arXiv preprint, 2024a.

A Answer formats and scoring

This appendix gives the implementation details of the per-format scoring rules summarised in Table 3 (Section 3.2). Single-select MCQ, multi-select MCQ, ranking, and tensor answers are scored deterministically; free-form answers (Level 4 only) are judged by an LLM panel under a fixed rubric. The three-step parser-then-judge cascade that handles non-canonical L1–L3 outputs is specified separately in Appendix D.

Single-select and multi-select MCQ. Single-select MCQ answers are parsed by extracting the first answer letter $\in \{\text{A, B, C, D}\}$ from the model's reply (tolerating "`C.`", "`C. some explanation`", etc.); a missing or malformed letter scores 0. Multi-select MCQ answers are length-four strings over $\{\text{T, F}\}$, scored positionally so each correctly classified statement contributes 0.25. We deliberately use the positional fraction rather than an all-or-nothing rule so that partial understanding (e.g. three of four statements correct) is credited.

Ranking. Ranking answers are parsed by extracting the first contiguous block of four characters in $\{\text{A, B, C, D}\}$ from the reply. The score is the fraction of positions where the predicted letter matches the gold permutation.

Tensor. Tensor answers are parsed by preferring the last bracketed block of the form "$[v_1, \ldots, v_n]$" in the model's reply that matches the expected length, falling back to the last n numerical tokens in the text. This avoids misreading reasoning intermediates (timestamps, partial computations) as the final answer. Each scalar is checked against a per-channel tolerance margin set at template-definition time, scaled to the natural unit of the channel (e.g. a few millimetres for end-effector positions, a few percent of the full-scale range for joint torques). The final score is $\frac{1}{n}\sum_{i=1}^{n} \mathbb{1}[|\hat{v}_i - v_i^\star| \le m_i]$, where $v_i^\star$ and m_i are the gold value and tolerance for the i-th element. When no per-channel margin is specified the comparison reduces to exact equality.

Free-form. Free-form answers are restricted to the two Level 4 question types — troubleshooting (identify root cause and corrective protocol) and optimization (identify misconfigured parameter values and corrective protocol) — and are scored by LLM-as-judge voting. Three frontier LLMs each independently cast a verdict in $\{\textbf{Wrong}, \textbf{Neutral}, \textbf{Correct}\}$ mapped to $\{0, 0.5, 1\}$, scoring the reply against the gold answer (root cause and corrective protocol for troubleshooting; misconfigured parameter values and corrective protocol for optimization). The final per-question score is the majority vote across the three judges, with three-way ties resolved to 0.5. Using a panel rather than a single judge limits the impact of any individual judge's idiosyncrasies; the rubric definition (gold root cause, gold protocol, what counts as "covering" the protocol) is fixed up front so the three judges score against the same ground-truth specification.

Aggregation. Per-question scores are averaged uniformly within a template, within a level, and overall. We do *not* reweight by answer format or by question difficulty, so the per-level numbers in Table 7 reflect the actual mix of formats used at that level (cf. Table 8).

B Question template inventory

Table 8 lists the number of question templates currently implemented per level, broken down by answer format. Several Level 1 templates are reinstantiated at Level 2 on anomalous episodes; those are counted only at the level where they are first introduced, so per-level rows and the overall total reflect unique templates.

Add details about how questions are marked. Also, research how to show questions are answerable

C Noise-substitution check

To probe whether the model scores in Table 7 reflect actual reading of the time-series context or merely priors over question templates and option positions, we constructed a noise-substituted counterpart of the dataset and re-ran every model against it.

Table 8: Number of unique question templates per level, by answer format. Templates reused across levels (e.g. Level 1 templates reapplied at Level 2 with anomalous instances) are counted once at the level where they are first introduced.

Level	MC single-select	MC multi-select	Ranking	Tensor / Numerical	Free-form	Total
1 (State)	1	2	1	2	0	6
2 (Intervention)	0	2	1	2	0	5
3 (Counterfactual)	0	2	1	2	0	5
4 (Decision)	0	0	2	0	2	4
Total	1	6	5	6	2	**20**

Construction. For each of the 400 generated Q&A pairs (100 per level), we duplicate the item and rewrite only the numerical time-series content. For each numeric *float* feature we preserve the original first value and substitute every subsequent value with the cumulative sum of independent Gaussian increments:

$$v'_0 = v_0, \qquad v'_{t+1} = v'_t + \mathcal{N}(0, \sigma^2), \qquad \sigma = 0.5,$$

where σ is shared across all features. The same substitution is applied to time-series chunks embedded in option strings (Level 1 severity-ranking, Level 2/3 chunk-ranking).

Expected behaviour. A model that scores by genuinely reading the time series should drop sharply on the noise-substituted variant, because the substitution destroys the signal-trajectory information that the gold answer hinges on. A model that scores by template-level priors (e.g. guessing the most plausible MC option given the question wording) or by option-position bias should be roughly invariant, since the question text and options are unchanged.

Result. Table 9 reports the side-by-side mean rubric scores. Three patterns emerge.

1. On Level 1 every model is essentially flat or improves slightly under substitution. This is consistent with L1 templates being substantially answerable from the question wording, the option set, and the still-present discrete schema fields (`task_phase`, `fault_label`). It also flags L1 as the level where overlap between LLM scores and a random/regression baseline is most expected.
2. On Level 2 and Level 3 every model drops, often substantially: DeepSeek and Mistral lose 9–11 points on L3, and gpt-5.1 loses 7.7 on L2. The L3 drops are particularly informative because L3 templates ask counterfactual "what would have happened" questions where the answer hinges on the actual signal trajectory rather than on a discrete annotation.
3. On Level 4, gpt-5.1's 7.6% original score collapses to 0.5% under noise substitution — a $15\times$ drop. Inspection of the per-item rubric notes shows that gpt-5.1's L4 mass on the original data came almost entirely from correctly emitting "no anomaly is present" on truly nominal episodes; once the time series is replaced by Gaussian noise, the same nominal episodes look anomalous to the model and it now hallucinates faults on them, losing its only mode of scoring. The other three models score ≤ 0.005 in either condition and have no comparable mode to lose.

D Evaluation protocol: the three-step scoring cascade

This appendix specifies the implementation of the three-step scoring cascade summarised in Section 5.4, mirroring the per-format definitions in Section 3.2. Every Q&A item carries an `answer_format` tag, a `ground_truth` field, and (for numerical and tensor formats) an `acceptance_bounds` block specifying the per-question margin. The cascade dispatches on `answer_format`: free-form is handled separately (judge-only); every other format goes through the three steps below.

1. **Parse** $r_1 \leftarrow \texttt{parser}_{\text{fmt}}(\text{prediction}, \text{ground\_truth})$.
   If r_1.`provenance` $\neq$ `unparseable`, return r_1.

Table 9: Noise-substitution check: measured mean scores (%) on the original FactoryBench Q&A pairs (Orig) vs. the noise-substituted variant (Sub) constructed by replacing every numerical time-series value with cumulative Gaussian increments seeded at the original first value ($\sigma = 0.5$). Integer-valued schema fields are preserved. Questions, options, and gold answers are identical between the two runs. Δ = Sub − Orig; large negative values are evidence that the model was reading the time series rather than the question alone.

	L1			L2			L3			L4		
Method	Orig	Sub	Δ	Orig	Sub	Δ	Orig	Sub	Δ	Orig	Sub	Δ
Claude Haiku 4.5	19.0	19.5	+0.5	25.5	26.7	+1.2	6.3	4.4	-1.9	0.5	0.0	-0.5
DeepSeek V3.1	16.5	15.0	-1.5	23.7	19.5	-4.2	24.7	15.7	-9.0	3.5	4.0	+0.5
gpt-5.1	24.0	24.0	0.0	28.7	21.0	-7.7	22.0	16.0	-6.0	7.6	0.5	-7.1
Mistral-Large-3	22.0	25.0	+3.0	22.7	21.8	-0.9	26.7	15.8	-10.9	0.0	0.0	0.0

2. **Extract** $e \leftarrow$ `judge.extract`(fmt, prediction) using a per-format extraction prompt. If $e =$ `NO_ANSWER`, return (score $= 0$, `provenance` $=$ `judge`).
3. **Re-parse** $r_2 \leftarrow \texttt{parser}_{\text{fmt}}(e, \text{ground\_truth})$. Return r_2 with `provenance` $=$ `judge` (recording that an LLM mediated the extraction).

The judge is never the source of the score for L1–L3: it only translates a non-canonical model output into a canonical string that the deterministic parser can read. This is what makes the protocol auditable.

Step 1 — Per-format parsers. The deterministic parsers consume the model's raw output string and emit a triple (score, parsed, `provenance`) where provenance is one of {`parser`, `judge`, `unparseable`}.

- *Multiple-choice single-select.* The parser extracts the final letter in $\{A, B, C, D\}$ from the response (after stripping intermediate options used in chain-of-thought) and compares against the ground-truth letter. Score is 1 on exact match, 0 otherwise.
- *Multiple-choice multi-select.* The model is asked for an n-character T/F string (one position per option statement). The parser reconstructs that string and compares position-by-position against the gold T/F vector; partial credit follows Section 3.2 (1 for exact match, 0.5 for one mismatch, 0 otherwise).
- *Ranking.* Output is parsed as a permutation of $\{A, B, C, D\}$ (or the shorter alphabet for 3-option templates). The parser reports both exact-match rate against the gold permutation and Kendall's τ rank correlation.
- *Numerical / Tensor.* The parser reads either a single scalar or an underscore-separated tensor. Each scalar is compared against the corresponding gold scalar with the per-question tolerance $|p - g| \leq \tau$ pulled from `acceptance_bounds.margin`. When that field is absent, the parser falls back to a $\pm 10\%$ band of $|g|$ (floored at 10^{-3}), matching the generator's default $k = 0.10 \cdot \text{window\_range}$. Tensor scores are the per-element mean.

If any parser cannot recover an answer in its canonical format, it returns `provenance` $=$ `unparseable` and the cascade falls through to Step 2.

Step 2 — Judge extraction. Each format has a dedicated extraction prompt that instructs the judge to recover the model's final canonical answer (ignoring intermediate options or reasoning steps) and to respond with only that canonical string, or with `NO_ANSWER` if the model refused. Examples:

- Single-select: "*Identify the model's FINAL chosen letter . . . Respond with ONLY a single letter (A, B, C, or D), or `NO_ANSWER`.*"
- Multi-select: "*Reconstruct the n-character T/F string. Respond with ONLY the string (e.g. `TFTF`), or `NO_ANSWER`.*"
- Ranking: "*Identify the model's FINAL ranking string. Respond with ONLY the n-letter permutation (e.g. `BDCA`), or `NO_ANSWER`.*"

- Numerical / Tensor: "*Respond with ONLY the number / underscore-separated values, or* `NO_ANSWER`."

The default extraction model is `gpt-5-mini`. The judge's role is purely transcription: it never sees the ground truth answer, and it never returns a score.

Step 3 — Re-parse and final verdict. The extracted string from Step 2 is fed back into the same Step 1 parser. The output of that re-parse is the final score, recorded with `provenance` = `judge` together with the judge's brief reason string, so any reader can later distinguish "model answered cleanly" (provenance = `parser`) from "model answered correctly but the judge had to canonicalise" (provenance = `judge`). If the re-parse still fails (judge returned something the parser cannot read), the score is logged as 0 with the same `judge` provenance.

Free-form scoring (out of cascade). Level 4 free-form templates bypass the three-step cascade and go directly to the judge. The judge is prompted with the question, the reference answer (root cause and/or remediation protocol), and the model's free-form response, and is asked to emit a single JSON line of the form `{"score": s, "reason": "..."}` with $s \in \{0, 0.5, 1\}$ following the rubric of Section 3.2: 1 for semantically equivalent (correct direction, signal, magnitude, time window), 0.5 for partially correct or imprecise but on-topic, 0 for incorrect, irrelevant, or refused. The same prompt and parser tolerate either a single-judge or three-judge voting backend (Section 3.2) without changes to the cascade.

E FactoryBench-Lite: distillation methodology and validation

F Details of the FactoryWave dataset

FactoryWave was collected on two one-armed platforms: the **Universal Robots UR3** (cobot; OnRobot Screwdriver and two-finger gripper variants) and the **KUKA KR10** (industrial arm). Each episode corresponds to one complete task cycle; episodes are recorded under four conditions (normal, fault, trajectory-optimization, and counterfactual), described below. Because state interpretation rather than policy quality is the object of study, motion is generated by a deterministic scripted controller. Every sample carries extensive labelling metadata, and the full native-rate sensor stream is logged without online feature selection.

F.1 Tasks and phase structure

Three canonical tasks are represented in the dataset: Pick-and-Place, Screwing, and Peg-in-Hole. All three are executed on both robots, using matched scripted controllers and task layouts so that cross-embodiment comparisons hold the task fixed. We structure each task as a deterministic sequence of phases (Tables 10, 11, 12); the phase index is written as a discrete label at every timestep, providing an interpretable segmentation that downstream models can condition on or that can be used to localize questions to a specific stage of the cycle. Within this fixed phase skeleton, we randomize as much of the episode as the task permits so that a question template that targets a given phase does not accidentally learn a fixed pose, timing, or trajectory. Pick and place locations are sampled uniformly from predefined workspace regions; phase-specific joint velocities, accelerations, and end-effector (EEF) rotations are drawn from bounded per-phase ranges; and approach angles to contact events are perturbed within the tolerances of the controller. This forces the Q&A pairs generated from FactoryWave to reflect phase-level reasoning rather than memorisation of a single scripted trajectory, which in turn makes them generalizable to a wide variety of trajectories and variation found in real recordings.

check eef is common to use

F.2 Episode metadata

Alongside the sensor stream, every episode carries a fixed set of metadata fields describing the robot, task, condition, payload, and end-effector configuration (Table 13). These fields are constant within an episode and are used both as filters during question generation (e.g. to restrict a template to a specific payload regime) and as provenance in the released Q&A pairs. This includes fault-specific fields (such as injection timestamp or physical offset).

Table 10: Task phases for Pick-and-Place.

Phase index	Phase name	Description
0	`above_pick`	EEF moves to a waypoint directly above the object
1	`descend`	EEF lowers toward the object
2	`settle`	Brief hold to let physics settle before grasping
3	`close`	Gripper closes around the object
4	`lift`	EEF lifts the grasped object off the surface
5	`move_xy`	Lateral transfer toward the bin
6	`lower`	EEF lowers into the bin
7	`open`	Gripper opens, object released
8	`retract`	EEF moves away from the bin
9	`return`	EEF returns to home configuration

Table 11: Task phases for Screwing.

Phase index	Phase name	Description
0	`approach`	EEF moves to a pre-contact waypoint above the fastener
1	`descend`	EEF lowers to engage the fastener
2	`screw`	Continuous downward force + wrist rotation to thread
3	`disengage`	Gripper opens / tool lifts off
4	`retract`	EEF returns to a safe height
5	`descend`	EEF lowers to engage the fastener (loosening pass)
6	`loosen`	Loosen the screw
7	`engage`	Gripper closes on the retrieved screw
8	`retract`	EEF returns to home position

Table 12: Task phases for Peg-in-Hole.

Phase index	Phase name	Description
0	`above_hole`	EEF moves above the hole and aligns
1	`insert`	EEF pushes peg downward into the hole
2	`disengage`	Gripper opens
3	`above_hole`	EEF rises back to the waypoint above the hole
4	`retract`	EEF returns to home position
5	`above_object`	EEF moves above the object and aligns with it
6	`engage`	Gripper closes
7	`above_object`	EEF rises back to the waypoint above the object
8	`retract`	EEF returns to home position

Table 13: Default per-episode metadata fields.

Field	Description
`episode_id`	Unique identifier for the episode
`robot_model`	Robot model (UR3, KUKA KR10)
`task`	Task (pick-and-place, screwing, peg-in-hole)
`condition`	Episode condition (normal, fault, trajectory-opt, counterfactual)
`fault_id`	Fault index for fault episodes; null otherwise
`weight_of_box`	Mass of the transported object in kg (pick-and-place only)
`shape_of_box`	Shape/dimensions of the transported object (pick-and-place only)
`position_of_box`	Initial XYZ position of the object on the table, where measurable
`gripper_model`	Gripper model and type (vacuum, two-finger, screwdriver)
`payload_mass_configured`	Payload mass configured at the controller (kg)
`payload_cog_configured`	Payload CoG offset configured at the controller (x, y, z in mm)
`tcp_offset_configured`	TCP position offset configured at the controller (x, y, z in mm)
`tcp_orientation_configured`	TCP orientation configured at the controller (rx, ry, rz in radians)

F.3 Fault taxonomy

FactoryWave includes 27 distinct fault types (Table 14). Each fault is paired with a plain-language description of the underlying physical or control-level mechanism and the injection procedure used to induce it during data collection. The last three columns indicate which tasks include the fault in the collected dataset; faults that apply to all three tasks (e.g. collisions with a shared workspace object, misconfigurations of the shared controller) are recorded under each applicable task with task-specific signal imprints.

add images of experiments

Table 14: Fault catalogue. Columns *PP*, *Scr*, *PiH* indicate whether the fault is included in the Pick-and-Place, Screwing, and Peg-in-Hole recordings.

Fault	Explanation	Injection	PP	Scr	PiH
Damaged screw thread	Screw thread physically damaged, preventing proper engagement during tightening.	Pre-damaged the screw thread with sandpaper.	×	✓	×
Missing screw	Tightening is attempted with no screw present.	Removed the screw from screwdriver before the cycle.	×	✓	×
Damaged plate thread	The threaded hole in the plate is damaged, preventing engagement.	Pre-damaged the plate hole with a metal screwdriver.	×	✓	×
Loosening phase	A counterclockwise loosening rotation replaces tightening; a normal phase with a distinct signal. It is counted here as an anomaly to match the fault schema of the AURSAD dataset Leporowski et al. [2022].	Reversed the rotation direction in the controller program to execute a counterclockwise loosening pass in place of tightening.	×	✓	×
Gripper activation failure	The vacuum gripper fails to activate and never picks up the box.	Disabled the gripper activation command in the robot program so the pick cycle completes without a grasp.	✓	×	×
Gripper release during motion	The gripper releases the payload mid-trajectory, causing an abrupt payload loss.	Triggered a programmatic gripper-release command at a scripted timestep mid-trajectory.	✓	×	×
Additional axis payload	A dead weight attached to one link increases inertia and gravity loading on all joints.	Bolted calibrated weights of varying mass to one robot link.	✓	✓	×
Collision with foam object	Contact with a soft foam block produces a brief TCP force spike without a protective stop.	Placed a foam cube directly in the programmed TCP trajectory.	✓	✓	✓
Unexpected payload weight	The transported box has a weight that deviates from the nominal payload configuration.	Swapped the nominal box for a known heavier or lighter one. This is done in a counterfactual context, ie. switching weights and asking about alternate weight scenario.	✓	×	×
Invalid gripping position	Timing error: the gripper closes after the arm has begun lifting, gripping the object off-centre.	Inserted a brief delay in the controller program so gripper closure lags the lift motion.	✓	×	×

Table 14 (cont.)

Fault	Explanation	Injection	PP	Scr	PiH
Unstable mounting platform	Base instability introduces low-frequency vibrations into the arm.	Placed 8 layers of towels under the base plate.	✓	×	×
Joint position limit violation	A joint moves outside its configured soft or hard position limits, triggering a safety stop.	Random waypoint slightly beyond the configured soft joint limit.	✓	×	×
TCP frame misconfiguration	TCP frame or mounting orientation misconfigured at the controller; gravity torques deviate.	Entered an incorrect TCP offset or mounting angle in the controller installation settings.	✓	✓	✓
Payload weight misconfiguration	Payload mass misconfigured while a tool or workpiece is physically attached.	Left the configured payload weight at multiple wrong mass configurations while a task-dependant gripper remained mounted.	✓	✓	×
External arm disturbance	A continuous external force pulls or pushes the TCP during motion.	Anchored an elastic rope between the bench frame and the tool flange.	✓	✓	✓
Mild payload CoG misconfiguration	Payload CoG specified at an incorrect offset from the tool flange; gravity compensation is wrong.	Entered a CoG offset in the payload configuration that differs from the physical tool CoG.	✓	✓	✓
Collision with hanging cable	A loose cable drags along a robot link or the tool flange during motion.	Suspended a loose cable across the trajectory at arm height.	✓	✓	✓
Collision with cardboard object	A cardboard carton in the trajectory provides moderate resistance; may trigger a protective stop.	Placed a free-standing or braced cardboard box in the programmed trajectory.	✓	✓	✓
Collision with rigid object	A rigid plastic block in the TCP path triggers an immediate protective stop.	Clamped a rigid plastic block in the programmed TCP path. Unlike the other collisions, this fault consistently causes safety stops.	✓	✓	✓
Peg insertion misalignment	Peg approaches the hole at an angular or lateral offset and jams against the rim.	Added a recorded lateral offset at the insertion approach waypoint.	×	×	✓
Hole obstruction	Foreign object or debris in the hole prevents full insertion.	Placed a small object (metal screw) inside the hole before insertion.	×	×	✓
Incorrect insertion depth	Insertion terminates at an incorrect Z depth due to a misconfigured waypoint.	Shifted the insertion endpoint waypoint Z by recorded offset from nominal.	×	×	✓
Peg surface contamination	Contamination or roughness on the peg raises insertion friction and produces stick-slip.	Applied dry chalk powder to the peg surface.	×	×	✓
Fixture displacement	Insertion fixture shifted laterally, breaking the match between programmed approach and actual hole.	Physically shifted the hole fixture by recorded offset without updating the robot program.	×	×	✓

Table 14 (cont.)

Fault	Explanation	Injection	PP	Scr	PiH
Self-collision	Arm configuration causes a link to contact the robot body or mounting fixture, triggering a protective stop.	Random waypoint that forces a self-colliding configuration or offset the mounting fixture into the trajectory.	✓	×	×
Missing box	Box absent from the pick position; the gripper closes on empty air.	Removed the box from the pick position before the episode started.	✓	×	×
Missing peg	Arm descends into the hole empty-handed; insertion produces no contact force.	Removed the peg from the gripper before the episode started.	×	×	✓

F.4 Trajectory-optimization episodes

Trajectory-optimization episodes capture the variability of executions that all complete the same task goal. Rather than following the fixed scripted path, the controller is driven between fixed waypoints using randomly sampled intermediate waypoint sequences, so that approach angles, intermediate poses, joint configurations, and motion speeds vary across episodes. This randomized experiment provide a proper source of comparisons for Level 4 optimization-centered templates.

rerecord trajectory episodes

F.5 Counterfactual episodes

Counterfactual pairs are constructed for the subset of faults in Table 14 whose injection moment can be controlled at a specific timestep rather than being present throughout the episode. For each such fault, the initial conditions of the physical setup (object identity and pose, payload configuration, controller settings) are held as constant as the platform allows, one baseline execution is recorded without any injection, and several fault executions are then recorded under the same initial conditions with the fault applied at a shared sampled timestep. Because the physical system is not perfectly reproducible, the pre-injection sensor trajectories of the baseline and the fault candidates diverge slightly; we retain, as the counterfactual partner of the baseline, the fault candidate whose pre-injection window is closest to the baseline in KL divergence, and discard the others. This yields an approximate counterfactual pair in which the early history is as matched as the real platform permits, while the post-injection divergence isolates the effect of the intervention. The approximation error introduced by this procedure, relative to the exact counterfactuals available in simulation, motivates the sim2real analysis in Section 5.3.

F.6 Signal schema

The UR3 is recorded through the `ur_rtde` Python interface at 125 Hz, yielding 136 per-sample signals grouped by semantic role (Table 15): controller *setpoints*, actual *effort / feedback* readings from joints and the TCP, *context / health* signals covering joint temperatures, voltages, and internal controller state, a small block of controller *status / mode* indicators, and digital/analog *I/O* lines. Six episode-level provenance columns (`episode_id`, `ctx_robot_type`, `ctx_task`, `ctx_fault_label`, `ctx_payload_kg`, `ctx_recording_date`) are appended per episode but are constant within an episode.

The KUKA KR10 is controlled through the KRC4 controller. We stream 44 per-sample signals at 83 Hz, grouped by semantic role in Table 16 using the same partition as the UR3 schema. A handful of signals that the UR3 exposes natively are unavailable on KSS 8.3, specifically joint velocities, commanded TCP pose, joint voltage, the tool-mounted accelerometer, and joint-level control outputs; TCP force/torque is also absent because no force sensor is fitted.

Table 15: UR3 signal groups recorded at 125 Hz via `ur_rtde` (excluding episode-level provenance columns).

Group	# signals	Contents
Setpoint (intent)	42	Target joint position, velocity, acceleration, current, torque; target TCP pose and speed.
Effort / feedback	48	Actual joint position, velocity, current, current-as-torque, joint control output; actual TCP pose, speed, and force/torque.
Context / health	33	Joint temperatures, joint voltages, joint modes, tool accelerometer, raw F/T wrench, main/robot voltage and current, momentum, speed scaling, execution time.
Status / mode	7	Timestamp, robot mode, robot status, safety mode, safety status bits, runtime state, configured payload mass.
I/O	6	Digital inputs/outputs, two analog inputs, two analog outputs.
Total	**136**	

Table 16: KUKA KR10 signal groups recorded at 83 Hz via RSI/KRL (excluding episode-level provenance columns).

Group	# signals	Contents
Setpoint (intent)	6	Commanded joint positions (degrees).
Effort / feedback	24	Actual joint positions (degrees), actual TCP pose (mm / degrees), per-axis motor current (% of max), and per-axis motor torque (Nm).
Context / health	11	Per-axis motor temperature (Kelvin), Cartesian acceleration on X, Y, Z and absolute magnitude (m/s^2), and speed override (%).
Status / mode	1	Controller process state (FREE / ACTIVE / STOP / END).
I/O	2	Digital inputs bitmask and digital outputs bitmask.
Total	**44**	

G Example question-answer pairs and question templates

Table 17 presents seventeen representative Q&A pairs spanning every causal level and every answer format the generator produces (single-select MCQ, multi-select MCQ, tensor, ranking, free-form). For readability we omit the underlying sensor context (tens to hundreds of time-series rows), truncate verbose boilerplate such as "Answer only with . . .", and abbreviate long option strings.

Table 17: Representative Q&A pairs from FactoryBench spanning all four causal levels and every answer format.

Level	Answer format	Prompt, options, and reference answer
L1	Tensor	**Q:** The robot is performing a screwing task. We want to isolate the screwing phase in the robot's time series. Assuming a fixed window length of 10 timesteps, at which timestamp should the window begin? **A:** `14` (accepted within ± 3)
L1	Single-select MCQ	**Q:** What changed between the two instances of robotic time series data? **Options:** **a.** different anomalous states (exactly one anomalous, or both but distinct) **b.** different tasks **c.** different robots **d.** same task, different phases **A:** `a`

continued on next page

Table 17 continued from previous page

Level	Answer format	Prompt, options, and reference answer
L1	Single-select MCQ	**Q:** What robot does this sensor data originate from? **Options:** **a.** Agile Robots Yu 5 Industrial **b.** KUKA KR 10 R1100-2 **c.** Universal Robots UR3 **A:** `c`
L1	Tensor	**Q:** Given the sensor stream below, what is the expected value of the position of joint 2 at T+6 ms? **A:** `102.7777`
L2	Tensor	**Q:** The sensor stream below is from a robot exhibiting a collision with a cardboard object. What is the expected value of the velocity of joint 3 at T+68 ms? **A:** `-0.272044` (accepted within ± 2.42)
L2	Tensor	**Q:** The sensor stream below is from a robot exhibiting a collision with a cardboard object. What are the expected values of joint positions at T+506 ms? **A:** `[17.884, -115.897, 121.049, -95.595, -90.258, 293.860]`
L2	Tensor	**Q:** Knowing that the robot suffers from a hole obstruction, we want to isolate the "rise back above the object" phase. Assuming a fixed window length of 16 timesteps, at which timestamp should the window begin? **A:** `202` (accepted within ± 3)
L2	Single-select MCQ	**Q:** Given the sensor data, determine what anomaly is present. **Options:** **a.** sustained command/measurement deviation **b.** no anomaly **c.** measured joint speeds drop abruptly with a TCP force spike (typical collision signature) **d.** isolated unrealistic force-sensor spike inconsistent with motion **A:** `c`
L3	Ranking	**Q:** Given the baseline stream and the counterfactual scenario where a collision foam object occurs at timestep 7366 ms, rank the four labeled signal segments in the order they would appear as the event manifests. **A:** `abdc`
L3	Multi-select MCQ	**Q:** Given the counterfactual scenario where a collision hanging cable occurs at timestep 8363 ms, select all statements that would apply. **Options:** **a.** motor current rises $\geq 33\%$ while speed stays $\leq 10\%$ of baseline (stall-like) **b.** contact-force spike $\geq 36\%$ above baseline **c.** tracking-error rise $\geq 34\%$ above pre-event mean **d.** at least one joint speed drops sharply ($\geq 31\%$) **A:** `FFFT`
L3	Tensor	**Q:** In the counterfactual scenario where a collision foam object occurs at timestep 7659 ms, what would be the expected value of the position of joint 2 at T+100 ms? **A:** `72.144939` (accepted within ± 2.77)
L4	Free form	**Q:** Given the sensor stream below, does the machine show signs of anomalous behavior? If yes, identify the most likely root cause and describe the steps you would take to fix it. **A:** No anomalous behavior detected; the machine is operating normally; no remediation is required.

continued on next page

Table 17 continued from previous page

Level	Answer format	Prompt, options, and reference answer
L4	Free form	**Q:** Given the sensor stream below, does the machine show signs of anomalous behavior? If yes, identify the most likely root cause and describe the steps you would take to fix it. **A:** **a.** Inspect the workspace and remove any soft obstructions from the programmed trajectory. **b.** If a protective stop occurred, acknowledge it in the robot controller and verify arm posture before resuming. **c.** Reduce speed if repeated false-positive collision detections occur.
L4	Free form	**Q:** Given the sensor stream below, does the machine show signs of anomalous behavior? If yes, identify the most likely root cause and describe the steps you would take to fix it. **A:** **a.** Stop the robot and update the payload configuration to include the additional axis weight. **b.** Set the correct CoG offset for the modified arm in the installation settings. **c.** Reduce motion accelerations and speeds to account for increased effective inertia. **d.** Run a slow-speed test cycle to verify dynamic loads stay within rated limits before resuming.
L4	Free form	**Q:** An engineer wants to increase the effectiveness and accuracy of this machine. Based on the sensor stream below, what operational changes or parameter adjustments could help achieve this? **A:** The payload mass is set to 0.0 kg but the actual payload weighs 1.5 kg. Update the payload mass in the installation settings to 1.5 kg.
L4	Free form	**Q:** An engineer wants to increase the effectiveness and accuracy of this machine. Based on the sensor stream below, what operational changes or parameter adjustments could help achieve this? **A:** The TCP offset is misconfigured at [0, 0.3, 55] instead of the correct [0, 0, 55]. Update the TCP position offset in the installation settings to [0, 0, 55].
L4	Free form	**Q:** An engineer wants to increase the effectiveness and accuracy of this machine. Based on the sensor stream below, what operational changes or parameter adjustments could help achieve this? **A:** The payload center of gravity is set to [25, 0, 20] but the correct value is [0, 0, 20]. Update the CoG offset in the installation settings to [0, 0, 20].

H Simulation setup

After Karim finishes the simulations, we fill this section up. Describe the simulation pipeline used for sim2real analysis: physics engine and version, robot URDFs, controller stack, supported tasks and fault applicators, randomization strategy, logging frequency and signal coverage, and how simulated episodes are aligned with the FactoryWave schema.

I Telemetry literacy vs. machine understanding

A model can reach a correct answer in two qualitatively different ways: (i) by *reading the telemetry* locating the right segment of the time series, identifying the active phase, recovering a numerical value within tolerance or (ii) by *reasoning about the machine* combining the read-off signal with knowledge of the controller, the task structure, the fault catalogue, and the manufacturer manual to predict an intervention, attribute a root cause, or produce a remediation protocol. The two capabilities can dissociate, as a model may parse the signal cleanly but fail to compose that information with machine-level knowledge, or hold strong priors over machine behaviour that compensate for shallow signal reading.

To probe this dissociation, we divide 83 Q&A pairs from the FactoryBench evaluation set into two diagnostic groups that target the two capabilities separately. The **telemetry literacy** group contains questions that can be answered from the attached time series alone, with no required knowledge of the robot. The **machine understanding** group contains questions that additionally require composing the telemetry with machine-level knowledge, such as predicting the effect of an intervention, attributing a fault to a root cause, or anticipating downstream consequences. Both groups are scored under the same per-format rubrics of Appendix A.

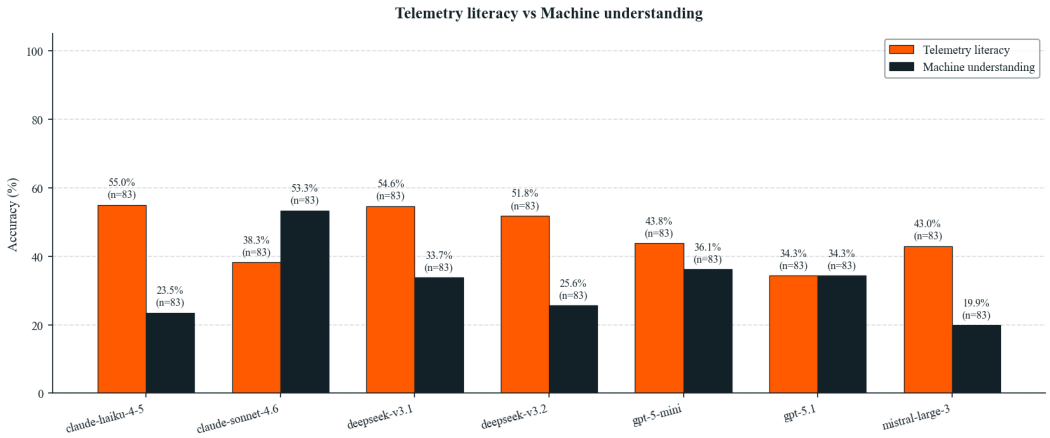

Figure 5: Per-model accuracy on the telemetry-literacy and machine-understanding diagnostic groups ($n = 83$ Q&A pairs per group). The gap between the two capabilities varies widely across models, with most models demonstrating strong performance in telemetry but limited comprehension of machine-level concepts.

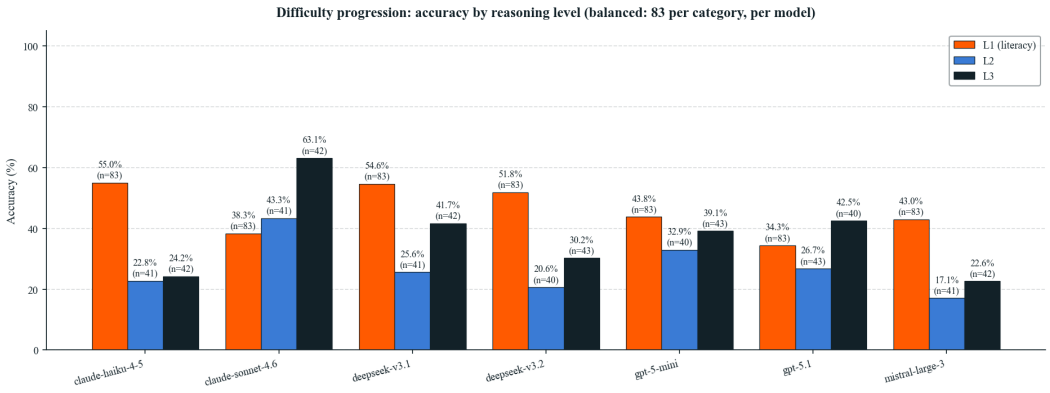

Figure 6: Per-model accuracy decomposed by reasoning level on the same balanced subset (83 Q&A pairs per model per category, with the 83 machine-understanding questions split between Level 2 and Level 3). Level 1 corresponds to telemetry-literacy questions; Levels 2 and 3 correspond to machine-understanding questions. The sharp L1→L2 drop seen in most models locates the bottleneck at the point where the telemetry must be combined with machine-level knowledge, rather than at the counterfactual rung (L3).

Figure 5 shows the difficulty of LLMs between these two groups. The y-axis gives the per-question accuracy averaged across all questions, and the x-axis gives the seven models evaluated. We observe that six of the seven models score higher on telemetry literacy than on machine understanding, with the gap exceeding 20 percentage points for `claude-haiku-4-5` (55.0 % vs. 23.5 %) and `mistral-large-3` (43.0 % vs. 19.9 %); these models can read the signal but stop short of using it to reason about the machine. In the other hand, `claude-sonnet-4-6` is the only model that inverts

the pattern (38.3 % literacy vs. 53.3 % understanding), suggesting it leans more on prior knowledge of robotic systems than on close reading of the attached telemetry, while `gpt-5.1` reaches parity at 34.3 % on both.

The two diagnostic groups are populated from different reasoning levels of the benchmark. The 83 telemetry-literacy questions are drawn entirely from **Level 1**, with questions that can be answered from the attached time series alone (phase identification, value read-off, segment ranking, anomaly spotting). The 83 machine-understanding questions are drawn from **Level 2** and **Level 3**, both of which target capabilities that go beyond literal signal reading: Level 2 requires composing the telemetry with knowledge of the robot's rated limits, safety modes, and controller behaviour, and Level 3 additionally requires counterfactual reasoning over alternative histories of the same episode. Decomposing the machine-understanding bar of Figure 5 into its L2 and L3 components (Figure 6) shows that the L1→L2 transition is where most models lose ground (e.g. `mistral-large-3` drops from 43.0 % to 17.1 %, `claude-haiku-4-5` from 55.0 % to 22.8 %), while L2→L3 is flat or slightly improving for several models. `claude-sonnet-4-6` is again the exception: accuracy rises monotonically with reasoning level (38.3 % → 43.3 % → 63.1 %), consistent with its inverted pattern in the headline figure.

J Random-context control: do the questions need the time series?

For each Q&A pair we ship a slice of recorded sensor data as the context the model is supposed to reason over. A natural soundness question is whether that slice actually carries the information needed to recover the ground-truth answer. If the answer is not derivable from the attached signals (because the relevant phase is not in the window, the sampling rate is too low, the discriminating features lie in channels we did not record, or the slice is simply too short), then performance reflects what the model can guess from its priors, not what it can read off the data.

To probe this, we run a control evaluation in which the prompt, options, acceptance bounds, and every other field of the Q&A pair are held fixed, but the attached time series is replaced with a random surrogate of the same shape: identical column schema, identical row count, and per-column values resampled from a distribution matched to the global mean and variance of that signal across the dataset. We re-run every model from the main panel under this random-context condition and compare per-level accuracy to the standard evaluation.

A large accuracy drop on a given template confirms that the recorded slice was informative enough for the model to answer from the data. A small drop or none at all means the slice is not the deciding factor: the model either had enough prior to answer without it or the data simply does not contain the discriminating signal at the resolution and length we provide. We use this to flag templates whose context windows need to be widened, resampled, or extended to additional channels before they are reported as headline accuracy numbers.

Fill once the random-context evaluation pass completes. Include per-template accuracy under the real and random-context conditions for every model in the main panel, plus the absolute and relative drop. Flag any template whose drop is below a threshold (proposed: 5 percentage points) as a candidate for redesign or exclusion.

K Causal schema (SCE): full details

This appendix gives the long-form definition of the Setpoint–Context–Effort + Feedback (SCE) schema used to organise every time-series channel in FactoryBench, together with the residual-based fault definition that the schema enables. The motivation is summarised in Section 3.3; here we cover the per-group channel breakdown, the residual formalism, and the per-robot calibration that makes faults computable rather than imputed.

The schema groups every recorded channel into one of three causal roles. **Setpoint** captures the controller's commanded target: per-axis position, velocity, and acceleration setpoints emitted by the trajectory generator. **Context** captures physical and configured conditions affecting behaviour, split into *static* context (episode-level metadata such as payload mass, payload centre of gravity, TCP offset, gripper model, and box / peg geometry) and *dynamic* context (per-sample channels such as joint temperatures, supply voltages, controller mode, and safety mode). **Effort + Feedback** captures

the machine's measured response: motor currents and torques (effort), measured joint positions and TCP pose (feedback), tool accelerometer, and acoustic emission where available. The exact per-robot channel mapping appears in the signal-group tables in this appendix (UR3 and KUKA KR10).

This split enables a single operational definition of a fault that applies across machines and tasks. Under healthy operation, the measured Effort + Feedback response $\mathbf{y}_t$ is by definition a function of the Setpoint command $\mathbf{S}$ and the Context $\mathbf{C}_t$:

$$\mathbf{y}_t = f(\mathbf{S}, \mathbf{C}_t),$$

where f represents the nominal plant dynamics (inverse-dynamics-plus-controller transfer) calibrated per robot. Faults manifest as structured residuals from this expected baseline: $\mathbf{y}_t - f(\mathbf{S}, \mathbf{C}_t)$ forms a clear, fault-specific spatiotemporal pattern. Because every episode supplies the paired $(\mathbf{S}, \mathbf{C}_t, \mathbf{y}_t)$ streams, this residual is computable directly from the data, giving ground truth that does not have to be hand-imputed. Figure 7 summarises the causal relationships between the three groups.

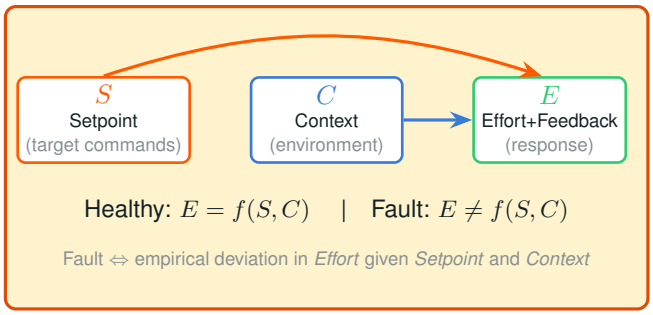

Figure 7: Physical causal schema. Setpoint commands S and contextual variables C jointly determine the measured Effort + Feedback response E under healthy operation. Faults manifest as residuals $E - f(S, C)$ from this nominal mapping.